

CSCI 423

Introduction to Algorithms



DR. RAAFAT ELFOULY

Useful summation formulas and rules

2

$$\sum_{l \leq i \leq u} 1 = 1 + 1 + \cdots + 1 = u - l + 1$$

In particular, $\sum_{1 \leq i \leq n} 1 = n - 1 + 1 = n \in \Theta(n)$

$$\sum_{1 \leq i \leq n} i = 1 + 2 + \cdots + n = n(n+1)/2 \approx n^2/2 \in \Theta(n^2)$$

$$\sum_{1 \leq i \leq n} i^2 = 1^2 + 2^2 + \cdots + n^2 = n(n+1)(2n+1)/6 \approx n^3/3 \in \Theta(n^3)$$

$$\sum_{0 \leq i \leq n} a^i = 1 + a + \cdots + a^n = (a^{n+1} - 1)/(a - 1) \text{ for any } a \neq 1$$

In particular, $\sum_{0 \leq i \leq n} 2^i = 2^0 + 2^1 + \cdots + 2^n = 2^{n+1} - 1 \in \Theta(2^n)$

$$\sum (a_i \pm b_i) = \sum a_i \pm \sum b_i \quad \sum c a_i = c \sum a_i \quad \sum_{l \leq i \leq u} a_i = \sum_{l \leq i \leq m} a_i + \sum_{m+1 \leq i \leq u} a_i$$



Algorithm Efficiency and Big-O

Algorithm Efficiency and Big-O

- Getting a precise measure of the performance of an algorithm is difficult
- Big-O notation expresses the performance of an algorithm as a function of the number of items to be processed
- This permits algorithms to be compared for efficiency
- For more than a certain number of data items, some problems cannot be solved by any computer

Linear Growth Rate

- If processing time increases in proportion to the number of inputs n , the algorithm grows at a linear rate

```
public static int search(int[] x, int target) {  
    for(int i=0; i < x.length; i++) {  
        if (x[i]==target)  
            return i;  
    }  
    return -1; // target not found  
}
```

Linear Growth Rate

- If processing time grows at the same rate as the number of inputs, the growth rate is linear

- If the target is not present, the for loop will execute `x.length` times
- If the target is present the for loop will execute (on average) $(x.length + 1) / 2$ times
- Therefore, the total execution time is directly proportional to `x.length`
- This is described as a growth rate of order n OR
- $O(n)$

```
public static int search(int[] x, int target) {  
    for(int i=0; i < x.length; i++) {  
        if (x[i]==target)  
            return i;  
    }  
    return -1; // target not found  
}
```

Quadratic Growth Rate

- If processing time is proportional to the square of the number of inputs n , the algorithm grows at a quadratic rate

```
public static boolean areUnique(int[] x) {  
    for(int i=0; i < x.length; i++) {  
        for(int j=0; j < x.length; j++) {  
            if (i != j && x[i] == x[j])  
                return false;  
        }  
    }  
    return true;  
}
```

Quadratic Growth Rate (cont.)

- If processing time is number of inputs n , rate

- The for loop with i as index will execute $x.length$ times
- The for loop with j as index will execute $x.length$ times
- The total number of times the inner loop will execute is $(x.length)^2$
- The growth rate has an order of n^2 or
- $O(n^2)$

```
public static boolean areUnique(int[] x) {  
    for(int i=0; i < x.length; i++) {  
        for(int j=0; j < x.length; j++) {  
            if (i != j && x[i] == x[j])  
                return false;  
        }  
    }  
    return true;  
}
```


Big-O Notation

- The $O()$ in the previous examples can be thought of as an abbreviation of "order of magnitude"
- A simple way to determine the big-O notation of an algorithm is to look at the loops and to see whether the loops are nested
- Assuming a loop body consists only of simple statements,
 - ▣ a single loop is $O(n)$
 - ▣ a pair of nested loops is $O(n^2)$
 - ▣ a nested pair of loops inside another is $O(n^3)$
 - ▣ and so on . . .

Formal Definition of Big-O

- Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}  
  
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}  
  
Simple Statement 6  
Simple Statement 7  
...  
Simple Statement 30
```

Formal Definition of Big-O (cont.)

- Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}  
  
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}  
  
Simple Statement 6  
Simple Statement 7  
...  
Simple Statement 30
```

**This nested loop
executes a *Simple
Statement* n^2 times**

Formal Definition of Big-O (cont.)

- Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}
```

```
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}
```

Simple Statement 6

Simple Statement 7

...

Simple Statement 30

**This loop executes 5
Simple Statements n times
($5n$)**

Formal Definition of Big-O (cont.)

- Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}
```

```
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}
```

Simple Statement 6

Simple Statement 7

...

Simple Statement 30



**Finally, 25 Simple
Statements are executed**

Formal Definition of Big-O (cont.)

- Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}  
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}  
Simple Statement 6  
Simple Statement 7  
...  
Simple Statement 30
```

We can conclude that the relationship between processing time and n (the number of data items processed) is:

$$T(n) = n^2 + 5n + 25$$

Formal Definition of Big-O (cont.)

- In terms of $T(n)$,

$$T(n) = O(f(n))$$

- There exist
 - ▣ two constants, n_0 and c , greater than zero, and
 - ▣ a function, $f(n)$,
- such that for all $n > n_0$, $cf(n) = T(n)$
- In other words, as n gets sufficiently large (larger than n_0), there is some constant c for which the processing time will always be less than or equal to $cf(n)$
- $cf(n)$ is an upper bound on performance

Formal Definition of Big-O (cont.)

- The growth rate of $f(n)$ will be determined by the fastest growing term, which is the one with the largest exponent

- In the example, an algorithm of

$$O(n^2 + 5n + 25)$$

is more simply expressed as

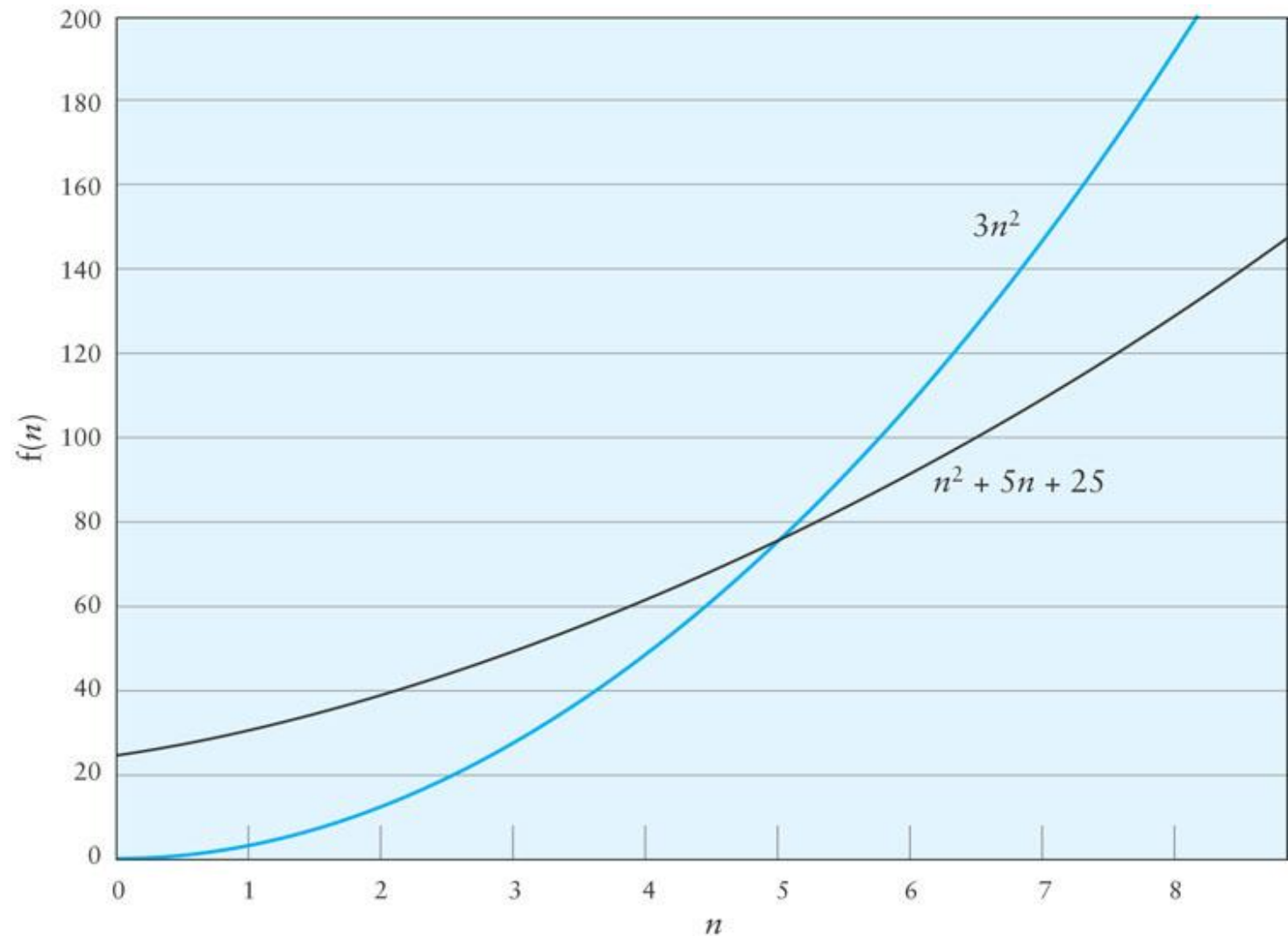
$$O(n^2)$$

- In general, it is safe to ignore all constants and to drop the lower-order terms when determining the order of magnitude

Big-O Example 1

- Given $T(n) = n^2 + 5n + 25$, show that this is $O(n^2)$
- Find constants n_0 and c so that, for all $n > n_0$, $cn^2 > n^2 + 5n + 25$
 - ▣ Find the point where $cn^2 = n^2 + 5n + 25$
 - ▣ Let $n = n_0$, and solve for c
$$c = 1 + 5/n_0 + 25/n_0^2$$
- When n_0 is $5(1 + 5/5 + 25/25)$, c is 3
- So, $3n^2 > n^2 + 5n + 25$ for all $n > 5$
- Other values of n_0 and c also work

Big-O Example 1 (cont.)



Big-O Notation (cont.)

- You must also examine the *number of times* a loop is executed

```
for(i=1; i < x.length; i *= 2) {  
    // Do something with x[i]  
}
```

How many times?

Big-O Notation (cont.)

- You must also examine the *number of times* a loop is executed

```
for(i=1; i < x.length; i *= 2) {  
    // Do something with x[i]  
}
```
- The loop body will execute $k-1$ times, with i having the following values:
 $1, 2, 4, 8, 16, \dots, 2^k$
until 2^k is greater than `x.length`
- Since $2^{k-1} = \text{x.length} < 2^k$ and $\log_2 2^k$ is k , we know that $k-1 = \log_2(\text{x.length}) < k$
- Thus we say the loop is $O(\log n)$ (in analyzing algorithms, we use logarithms to the base 2)
- Logarithmic functions grow **slowly** as the number of data items n increases

Big-O Example 2

□ Consider the following loop

```
for (int i = 0; i < n; i++) {  
    for (int j = i + 1; j < n; j++) {  
        3 simple statements  
    }  
}
```

How many statements will be executed?

Big-O Example 2

- Consider the following loop

```
for (int i = 0; i < n; i++) {  
    for (int j = i + 1; j < n; j++) {  
        3 simple statements  
    }  
}
```

- $T(n) = 3(n - 1) + 3(n - 2) + \dots + 3$

- Factoring out the 3,

$$3(n - 1 + n - 2 + n - 3 + \dots + 1)$$

Big-O Example 2

- Consider the following loop

```
for (int i = 0; i < n; i++) {  
    for (int j = i + 1; j < n; j++) {  
        3 simple statements  
    }  
}
```

- $T(n) = 3(n - 1) + 3(n - 2) + \dots + 3$

- Factoring out the 3,

$$3(n - 1 + n - 2 + n - 3 + \dots + 1)$$

- $1 + 2 + \dots + n - 1 = ???$

Big-O Example 2

- Consider the following loop

```
for (int i = 0; i < n; i++) {  
    for (int j = i + 1; j < n; j++) {  
        3 simple statements  
    }  
}
```

- $T(n) = 3(n - 1) + 3(n - 2) + \dots + 3$

- Factoring out the 3,

$$3(n - 1 + n - 2 + n - 3 + \dots + 1)$$

- $1 + 2 + \dots + n - 1 = (n \times (n-1))/2$

- Number of statements: $3 \times (n \times (n-1))/2$

Formal Definition of Big-O (cont.)

- In terms of $T(n)$,

$$T(n) = O(f(n))$$

- There exist

- ▣ two constants, n_0 and c , greater than zero, and
- ▣ a function, $f(n)$,

- such that for all $n > n_0$, $cf(n) = T(n)$

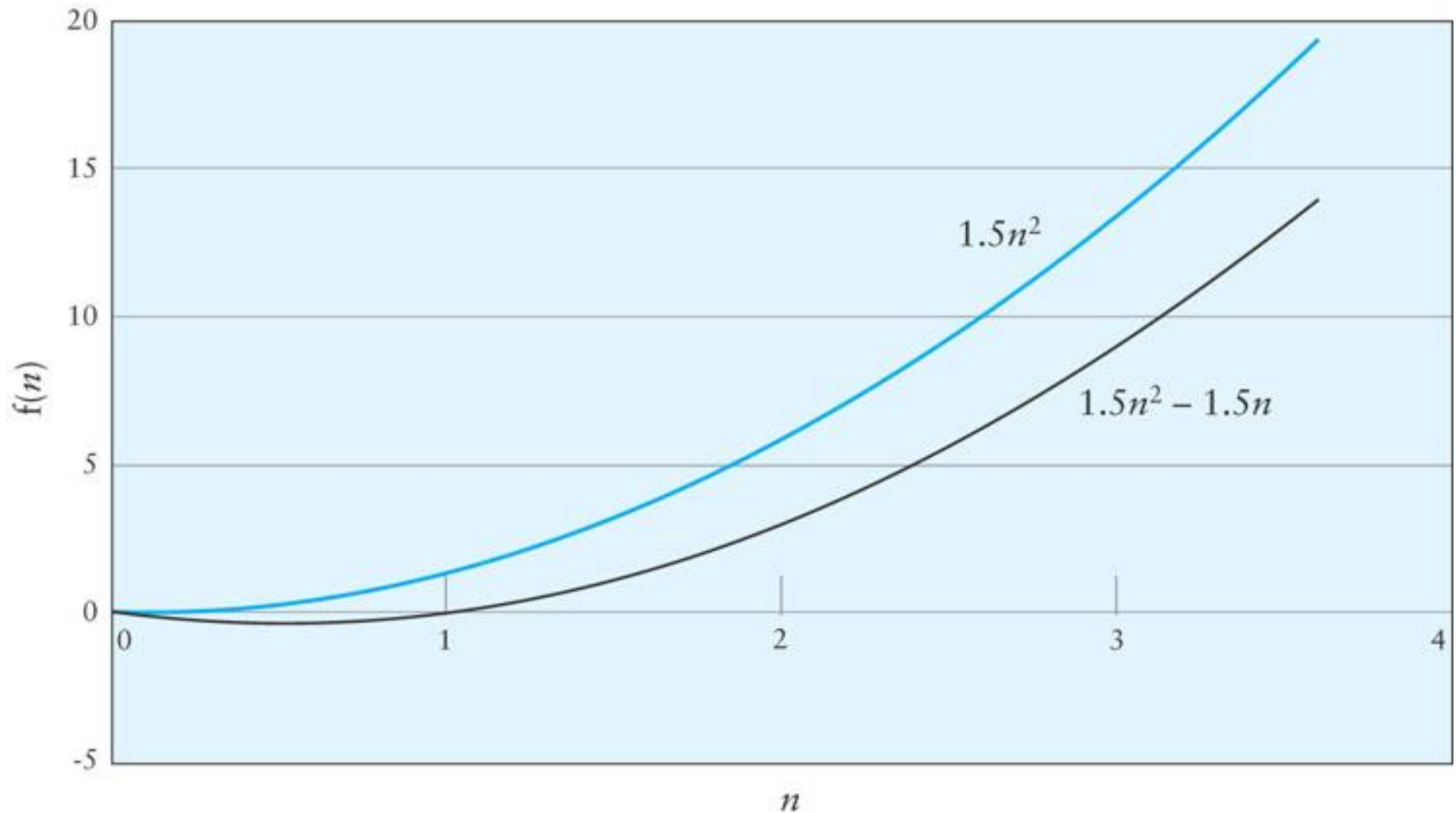
- In other words, as n gets sufficiently large (larger than n_0), there is some constant c for which the processing time will always be less than or equal to $cf(n)$

- $cf(n)$ is an upper bound on performance

Big-O Example 2 (cont.)

- Therefore $T(n) = 1.5n^2 - 1.5n$
- When $n = 0$, the polynomial has the value 0
- For values of $n > 1$, $1.5n^2 > 1.5n^2 - 1.5n$
- Therefore $T(n)$ is $O(n^2)$ when n_0 is 1 and c is 1.5

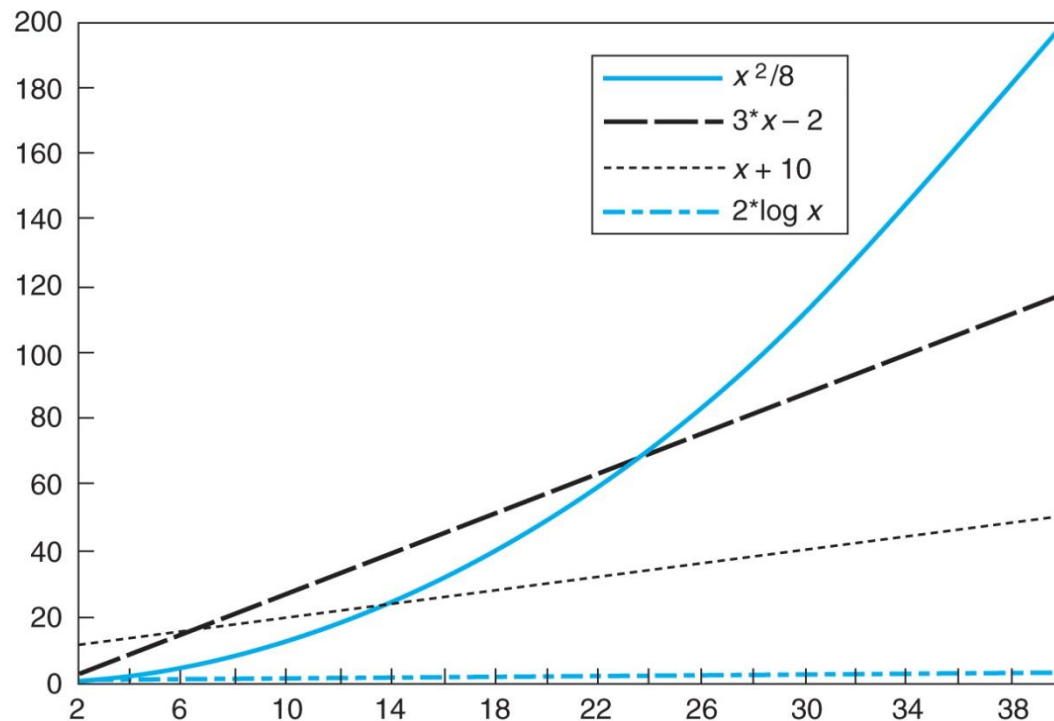
Big-O Example 2 (cont.)



Rate of Growth

28

- The rate of growth of a function determines how fast the function value increases when the input increases



■ **FIGURE 1.1**
Graph of four
functions

Rate of Growth

29

- If algorithm A does x^3 operations on an input of size x and algorithm B does x^2 operations, algorithm B is more efficient
- Because of the relative growth rate, we will consider x^3 compared to x^2

Common Growth Rates

Big-O	Name
$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n \log n)$	Log-linear
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(2^n)$	Exponential
$O(n!)$	Factorial

Rate of Growth

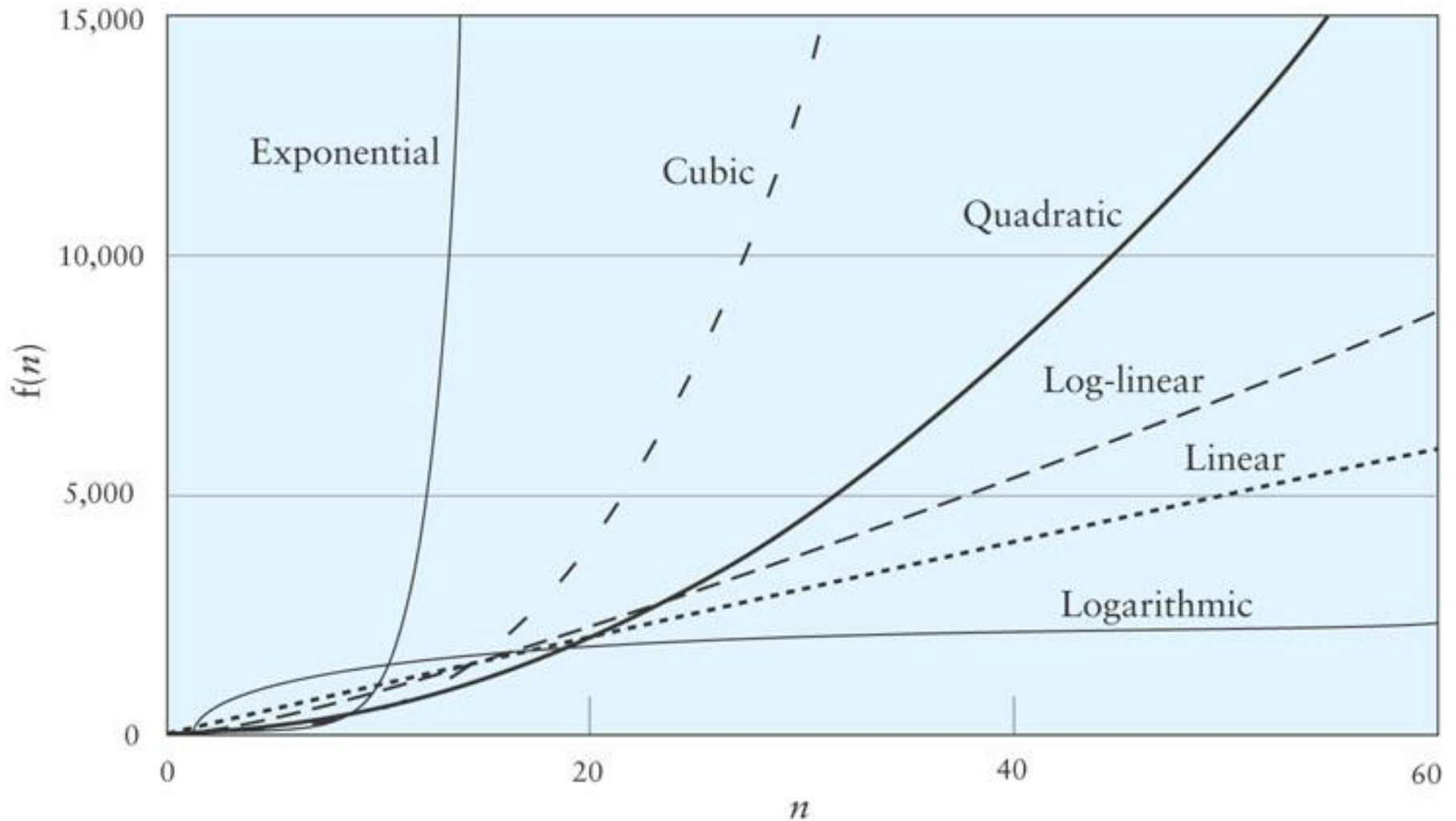
31

■ **FIGURE 1.2**

Common algorithm classes (The bottom six rows of the last column are approximations.)

	$\lg n$	n	$n \lg n$	n^2	n^3	2^n
1	0.0	1.0	0.0	1.0	1.0	2.0
2	1.0	2.0	2.0	4.0	8.0	4.0
5	2.3	5.0	11.6	25.0	125.0	32.0
10	3.3	10.0	33.2	100.0	1000.0	1024.0
15	3.9	15.0	58.6	225.0	3375.0	32768.0
20	4.3	20.0	86.4	400.0	8000.0	1048576.0
30	4.9	30.0	147.2	900.0	27000.0	1073741824.0
40	5.3	40.0	212.9	1600.0	64000.0	1099511627776.0
50	5.6	50.0	282.2	2500.0	125000.0	1125899906842620.0
60	5.9	60.0	354.4	3600.0	216000.0	1152921504606850000.0
70	6.1	70.0	429.0	4900.0	343000.0	1180591620717410000000.0
80	6.3	80.0	505.8	6400.0	512000.0	1208925819614630000000000.0
90	6.5	90.0	584.3	8100.0	729000.0	1237940039285380000000000000.0
100	6.6	100.0	664.4	10000.0	1000000.0	1267650600228230000000000000000.0

Different Growth Rates



Order of growth

33

- Most important: Order of growth within a constant multiple as $n \rightarrow \infty$
- Example:
 - How much longer does it take to solve problem of double input size?

Effects of Different Growth Rates

$O(f(n))$	$f(50)$	$f(100)$	$f(100)/f(50)$
$O(1)$	1	1	1
$O(\log n)$	5.64	6.64	1.18
$O(n)$	50	100	2
$O(n \log n)$	282	664	2.35
$O(n^2)$	2500	10,000	4
$O(n^3)$	12,500	100,000	8
$O(2^n)$	1.126×10^{15}	1.27×10^{30}	1.126×10^{15}
$O(n!)$	3.0×10^{64}	9.3×10^{157}	3.1×10^{93}

Algorithms with Exponential and Factorial Growth Rates

- Algorithms with exponential and factorial growth rates have an effective practical limit on the size of the problem they can be used to solve
- With an $O(2^n)$ algorithm, if 100 inputs takes an hour then,
 - ▣ 101 inputs will take 2 hours
 - ▣ 105 inputs will take 32 hours
 - ▣ **114 inputs will take 16,384 hours (almost 2 years!)**

Rate of Growth

36

■ **FIGURE 1.2**

Common algorithm classes (The bottom six rows of the last column are approximations.)

	$\lg n$	n	$n \lg n$	n^2	n^3	2^n
1	0.0	1.0	0.0	1.0	1.0	2.0
2	1.0	2.0	2.0	4.0	8.0	4.0
5	2.3	5.0	11.6	25.0	125.0	32.0
10	3.3	10.0	33.2	100.0	1000.0	1024.0
15	3.9	15.0	58.6	225.0	3375.0	32768.0
20	4.3	20.0	86.4	400.0	8000.0	1048576.0
30	4.9	30.0	147.2	900.0	27000.0	1073741824.0
40	5.3	40.0	212.9	1600.0	64000.0	1099511627776.0
50	5.6	50.0	282.2	2500.0	125000.0	1125899906842620.0
60	5.9	60.0	354.4	3600.0	216000.0	1152921504606850000.0
70	6.1	70.0	429.0	4900.0	343000.0	1180591620717410000000.0
80	6.3	80.0	505.8	6400.0	512000.0	1208925819614630000000000.0
90	6.5	90.0	584.3	8100.0	729000.0	1237940039285380000000000000.0
100	6.6	100.0	664.4	10000.0	1000000.0	1267650600228230000000000000000.0

Values of some important functions as $n \rightarrow \infty$

37

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Table 2.1 Values (some approximate) of several functions important for analysis of algorithms

Asymptotic order of growth

38

A way of comparing functions that ignores:
constant factors and **small input sizes**

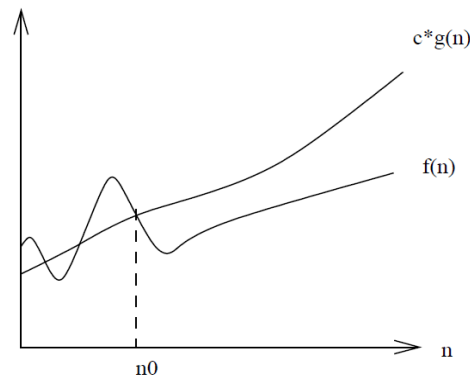
- $O(g(n))$: class of functions $f(n)$ that grow no faster than $g(n)$
- $\Theta(g(n))$: class of functions $f(n)$ that grow at same rate as $g(n)$
- $\Omega(g(n))$: class of functions $f(n)$ that grow at least as fast as $g(n)$



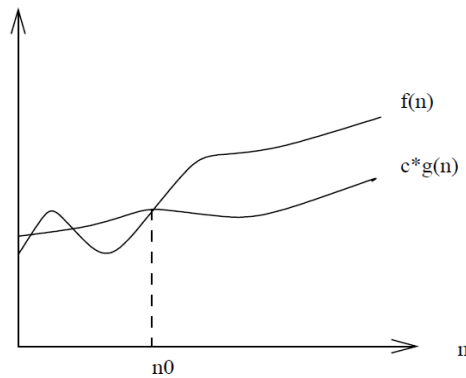
Formal Definitions

- $f(n) = O(g(n))$ if there are positive constants n_0 and c such that to the right of n_0 , the value of $f(n)$ always lies on or below $c \cdot g(n)$.
- $f(n) = \Omega(g(n))$ if there are positive constants n_0 and c such that to the right of n_0 , the value of $f(n)$ always lies on or above $c \cdot g(n)$.
- $f(n) = \Theta(g(n))$ if there exist positive constants n_0 , c_1 , and c_2 such that to the right of n_0 , the value of $f(n)$ always lies between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$ inclusive.

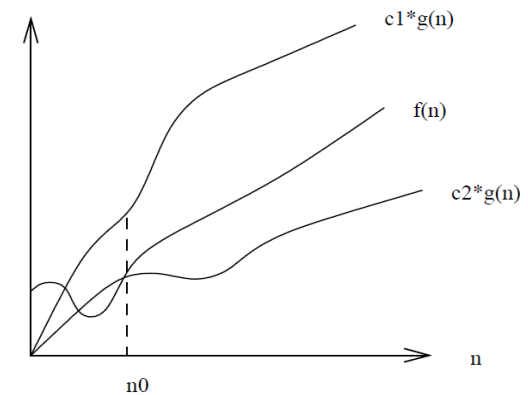
O , Ω , and Θ



(a)



(b)



(c)

The definitions imply a constant n_0 *beyond which* they are satisfied. We do not care about small values of n .

Big-oh

(41)

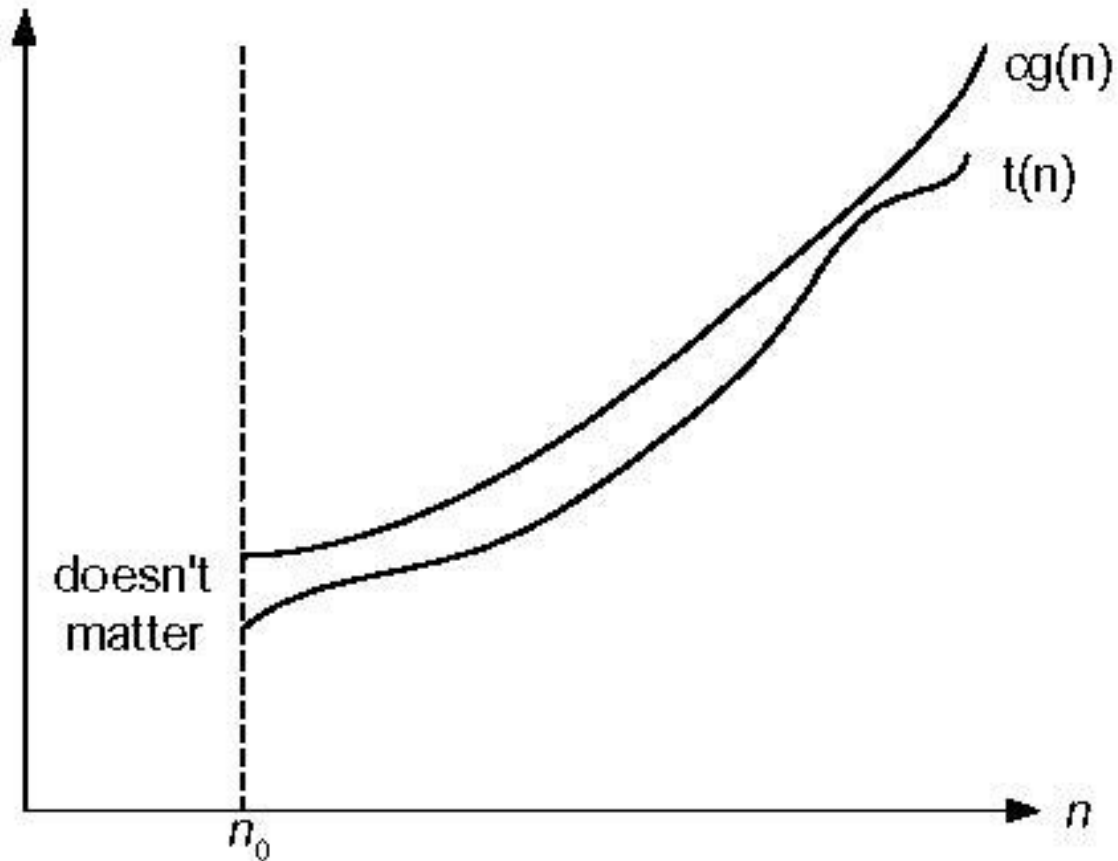


Figure 2.1 Big-oh notation: $t(n) \in O(g(n))$

Establishing order of growth using the definition

42

Definition: $f(n)$ is in $O(g(n))$ if order of growth of $f(n) \leq$ order of growth of $g(n)$ (within constant multiple), i.e., there exist positive constant c and non-negative integer n_0 such that

$$f(n) \leq c g(n) \text{ for every } n \geq n_0$$

Examples:

- $10n$ is $O(n^2)$
- $5n+20$ is $O(n)$